TECMILENIO

FASE II

Manual de despliegue en la nube del sistema inteligente de priorización de eventos de ciberseguridad

Materia:	*Gestión de Proyectos de Inteligencia Artificial*
Actividad:	*Fase II – Manuel de despliegue en la nube*
Alumno(a):	*Alejandro Bárcenas Godínez*
Matrícula:	*7182682*
Docente:	*Luis Ariel Vázquez Piña*
Fecha de entrega:	*[08 / 11 / 2026]*

Declaración de Uso de Inteligencia Artificial

El alumno debe indicar de manera honesta si utilizó herramientas de Inteligencia Artificial (IA) para el desarrollo de esta actividad. Completa la siguiente tabla:

Herramienta de IA utilizada	¿La usaste?	Propósito / Descripción de uso
ChatGPT / OpenAI	☒Sí ☐ No	Apoyo en la planificación del procedimiento de despliegue, revisión de comandos, documentación técnica y organización académica del manual. El despliegue en nube descrito es propuesto; la ejecución comprobada fue local con Docker.
Claude (Anthropic)	☐Sí ☒ No	
Gemini (Google)	☐Sí ☒ No	
Copilot (Microsoft)	☐Sí ☒ No	
Otra (especifica): Miro	☐Sí ☒ No	

Desarrollo de Contenido

1. Introducción

Este manual describe una ruta propuesta para desplegar en un servicio de nube administrado el prototipo académico de priorización de eventos de ciberseguridad de la Fase II. El sistema ya se construyó y ejecutó localmente con Docker Desktop, FastAPI, una interfaz web y un pipeline de regresión logística serializado con Joblib. El objetivo del manual es que una persona con acceso autorizado a la nube pueda reproducir la instalación, comprobar el servicio y revertirlo si es necesario.

El escenario de nube seleccionado es Azure Container Apps, con Azure Container Registry (ACR) como registro privado de imágenes. Es una propuesta de arquitectura, no un despliegue efectivamente realizado. Por ello, los comandos y verificaciones remotas se presentan como procedimiento a ejecutar; no se atribuyen resultados de nube que no se hayan obtenido.

2. Objetivo y Alcance

2.1 Objetivo General

Documentar un procedimiento técnicamente verificable para publicar la imagen Docker del prototipo en un servicio administrado, configurar su acceso y comprobar los endpoints GET /health y POST /predict sin modificar el modelo entrenado.

2.2 Alcance y Supuestos

Se contempla un entorno académico de demostración con un solo contenedor, tráfico de prueba y recursos mínimos. No se incluyen integración con SIEM, autenticación de usuarios finales, operación SOC, registro de decisiones humanas ni garantías de disponibilidad empresarial. La inferencia se basa en etiquetas de eventos y variables contextuales sintéticas; no es un sistema de detección autónoma de incidentes.

2.3 Validación Previa del Entorno Local

Antes de definir la estrategia de despliegue en la nube, se verificó el funcionamiento del prototipo en un entorno local mediante Docker Desktop y WSL 2.

Durante esta etapa se construyó satisfactoriamente la imagen `fase-ii-ciberseguridad:1.0` y se ejecutó el contenedor `fase-ii-app`, publicando el puerto 8001 del equipo anfitrión hacia el puerto 8000 del contenedor.

Posteriormente, se comprobó la disponibilidad del servicio mediante el endpoint `GET /health`, obteniendo el estado operativo. Asimismo, se verificó la integración entre la interfaz web, FastAPI y el modelo de clasificación mediante solicitudes de predicción.

Estas comprobaciones constituyen la base técnica para proponer el despliegue mediante Azure Container Registry y Azure Container Apps. Sin embargo, la ejecución validada corresponde exclusivamente al entorno local, por lo que el despliegue en Azure se presenta como un procedimiento propuesto y no como una implementación realizada.

3. Arquitectura y Estrategia de Despliegue

3.1 Arquitectura de Referencia

```
Usuario de prueba (HTTPS)
        |
Azure Container Apps (ingress HTTPS, puerto destino 8000)
        |
Contenedor FastAPI + frontend HTML + pipeline Joblib
        ^
Azure Container Registry (imagen versionada)
        ^
GitHub → estación de trabajo / CI autorizado → docker build / push
```

La imagen contiene app/main.py, app/static/index.html, modelo/modelo_priorizacion.joblib y las dependencias fijadas en requirements.txt. La API escucha en 0.0.0.0:8000 dentro del contenedor. El servicio administrado termina HTTPS y dirige el tráfico al puerto 8000; el puerto 8001 corresponde únicamente a la demostración local.

3.2 Elección de Contenedores Administrados

Azure Container Apps permite ejecutar contenedores sin administrar servidores virtuales. ACR centraliza las imágenes y sus etiquetas. Esta combinación reduce tareas de infraestructura para el ejercicio, pero requiere suscripción, permisos de creación de recursos, cuotas disponibles y revisión de costos. La elección es ilustrativa y no implica que Azure sea el único proveedor viable.

3.3 Riesgos y Decisiones de Seguridad

Riesgo	Medida propuesta
API pública sin autenticación	Usar entorno aislado; restringir acceso o añadir autenticación antes de uso externo
Costos y consumo	Establecer presupuesto/alertas; eliminar recursos al finalizar
Imagen vulnerable	Escanear dependencias e imagen; actualizar base y reconstruir
Credenciales de nube	No guardar claves en GitHub ni en la imagen; usar identidad administrada
Datos sensibles	Usar únicamente datos académicos sintéticos; no enviar eventos reales
Cambios no trazables	Versionar imagen con tag único y registrar digest

4. Requerimientos Técnicos

Elemento	Requisito / comprobación
Repositorio	Acceso de lectura a github.com/Aletz44/Fase-II-Ciberseguridad-ABG
Docker	Docker Desktop o Docker Engine operativo; contenedores Linux
Imagen base	python:3.13-slim, según Dockerfile del proyecto
Dependencias	numpy 2.1.3, pandas 2.2.3, scikit-learn 1.6.1, joblib 1.6.0, fastapi 0.162.0, uvicorn 0.54.0
Azure	Suscripción activa; permisos para grupo de recursos, ACR y Container Apps
Herramientas	Azure CLI con extensión containerapp; Git y terminal
Red	Salida HTTPS a repositorios y registro; endpoint HTTPS de aplicación
Recursos	Definir CPU/memoria según cuota y ensayo de carga; no se validó dimensionamiento en nube

Antes de ejecutar comandos de nube, confirmar la versión vigente de Azure CLI y la disponibilidad regional de Azure Container Apps. La sintaxis de opciones y permisos puede cambiar; se debe contrastar con la documentación oficial del proveedor.

5. Procedimiento de Despliegue Paso a Paso

5.1 Obtención del Código y Validación de la Estructura

```
git clone https://github.com/Aletz44/Fase-II-Ciberseguridad-ABG.git
cd Fase-II-Ciberseguridad-ABG
git status
```

Verificar la presencia de Dockerfile, requirements.txt, app/, modelo/, notebooks/ y README.md. El notebook documenta entrenamiento, pero no es necesario ejecutarlo para iniciar la API. No publicar secretos ni el entorno virtual .venv.

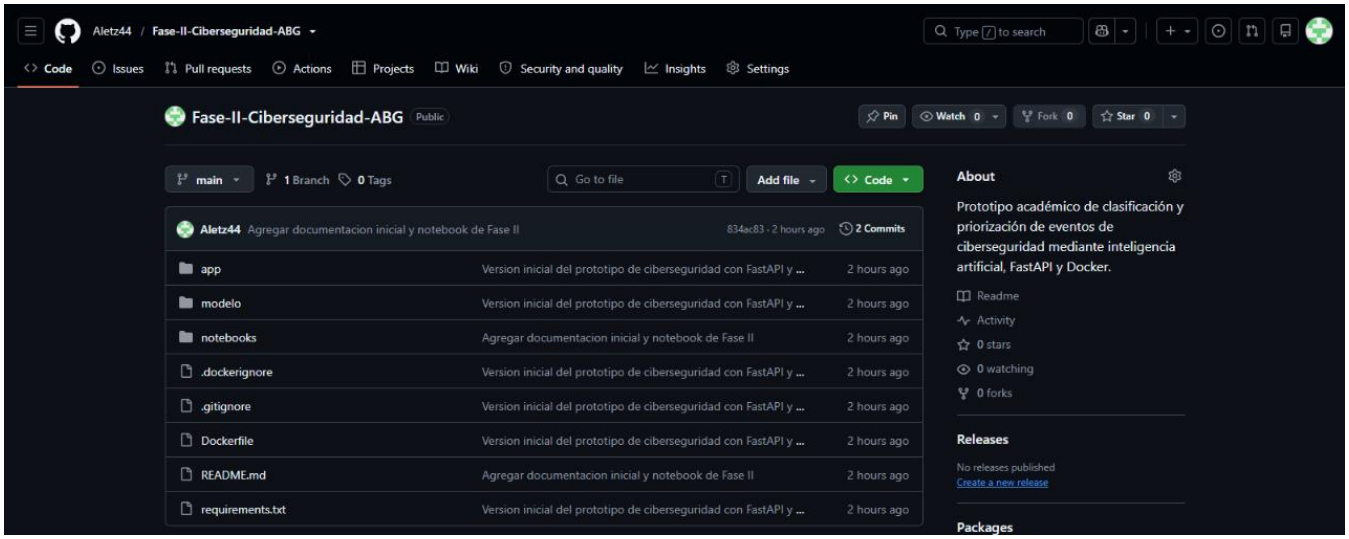


Figura 1. Repositorio GitHub actualizado con archivos de despliegue y README. Fuente: evidencia local del proyecto.

5.2 Construcción y Comprobación Local de la Imagen

```
docker build -t fase-ii-ciberseguridad:1.0 .
docker run -d --name fase-ii-prueba -p 8001:8000 fase-ii-ciberseguridad:1.0
docker ps
curl http://localhost:8001/health
```

Resultado esperado: el contenedor aparece en estado Up y GET /health devuelve estado operativo. Si ya existe un contenedor utilizando el puerto 8001, detenerlo o elegir otro puerto del host. Esta prueba fue realizada previamente en el equipo de desarrollo, pero una clonación limpia aún debe verificarse por separado.

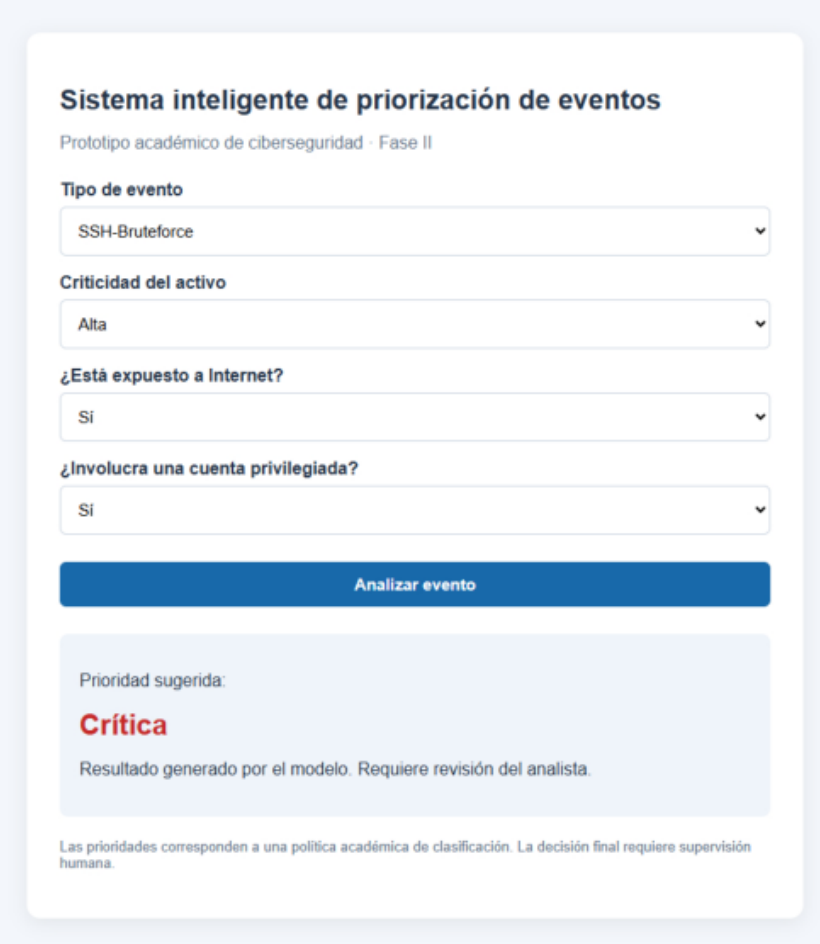


Figura 2. Aplicación ejecutada localmente mediante Docker (localhost:8001). Fuente: prueba del prototipo.

5.3 Preparación de la Sesión y los Recursos Azure

Los identificadores siguientes son ejemplos; sustituirlos por valores propios. El nombre de ACR debe ser globalmente único, usar solo caracteres permitidos y no contener información sensible. Ejecutar únicamente en una suscripción propia o autorizada.

```
az login
az account show
az extension add --name containerapp --upgrade
az provider register --namespace Microsoft.App
az provider register --namespace Microsoft.OperationalInsights

az group create --name rg-faseii-demo --location eastus
az acr create --resource-group rg-faseii-demo --name <ACR_UNICO> --sku Basic
```

5.4 Publicación de Una Imagen Versionada en ACR

```
az acr login --name <ACR_UNICO>
docker tag fase-ii-ciberseguridad:1.0 <ACR_UNICO>.azurecr.io/fase-ii-ciberseguridad:1.0
docker push <ACR_UNICO>.azurecr.io/fase-ii-ciberseguridad:1.0
```

Registrar el digest devuelto por el push. No utilizar la etiqueta latest como único identificador de entrega. La carga a ACR no se ha ejecutado en este proyecto; el comando constituye una instrucción reproducible para un entorno autorizado.

5.5 Creación del Entorno de Container Apps

```
az containerapp env create --name cae-faseii-demo --resource-group rg-faseii-demo --
location eastus
```

Para un entorno real, configurar acceso al registro mediante identidad administrada con permiso AcrPull, en lugar de incrustar contraseñas. La creación y asignación de esa identidad debe seguir la política de la suscripción y las instrucciones oficiales de Azure. El ejemplo siguiente presupone que dicha autorización ya se configuró.

5.6 Creación de la Aplicación Administrada

```
az containerapp create --name ca-faseii-demo --resource-group rg-faseii-demo --environment
cae-faseii-demo --image <ACR_UNICO>.azurecr.io/fase-ii-ciberseguridad:1.0 --target-port
8000 --ingress external --registry-server <ACR_UNICO>.azurecr.io --registry-identity system
```

Advertencia operativa: la opción --registry-identity system requiere que la identidad administrada tenga acceso de lectura al ACR; según el orden de aprovisionamiento puede ser necesario crear primero la aplicación/identidad y después configurar el registro e imagen. Si el comando falla por autorización, no habilitar el usuario administrador de ACR como solución permanente: configurar AcrPull y repetir siguiendo la guía oficial.

El ingreso externo publica la API hacia Internet. Para una práctica académica, limitar el acceso mediante controles de red o autenticación cuando sea posible y evitar información sensible. La interfaz y el endpoint /docs no deben considerarse seguros para producción por defecto.

5.7 Obtención de URL y Comprobación de Estado

```
az containerapp show --name ca-faseii-demo --resource-group rg-faseii-demo --query
properties.configuration.ingress.fqdn -o tsv
az containerapp logs show --name ca-faseii-demo --resource-group rg-faseii-demo --tail 50
```

Abrir https://<FQDN>/health y https://<FQDN>/docs. La primera ruta debe responder con estado operativo; la segunda debe presentar la documentación interactiva. En caso de error, revisar logs, puerto destino, imagen y permisos del registro. No se dispone de capturas de estas rutas en Azure porque no se realizó el despliegue remoto.

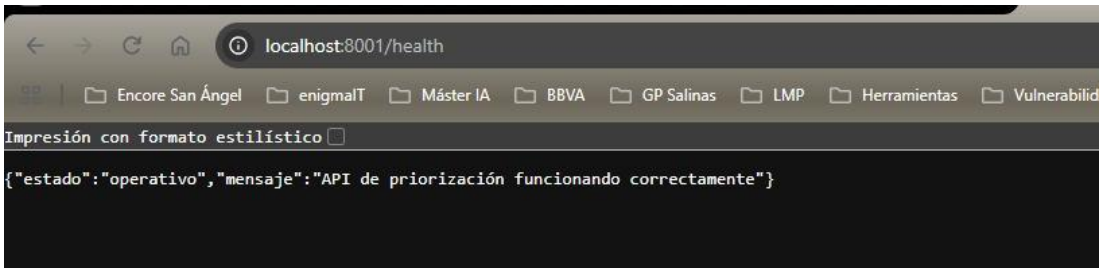


Figura 3. Comprobación local de GET /health en localhost:8001. No corresponde a una ejecución en Azure. Fuente: prueba local.

6. Pruebas de Aceptación

La aceptación del despliegue no debe basarse únicamente en que la imagen arranque. Se propone repetir los mismos casos de la API que se verificaron localmente, conservando entradas y resultados esperados.

Caso	Solicitud resumida	Criterio de aceptación
A01	GET /health	HTTP 200; estado operativo
A02	Benign / Baja / 0 / 0	HTTP 200; prioridad Baja
A03	Benign / Alta / 0 / 0	HTTP 200; prioridad Media
A04	Benign / Alta / 1 / 1	HTTP 200; prioridad Alta
A05	SSH-Bruteforce / Alta / 1 / 1	HTTP 200; prioridad Crítica
A06	Exposicion_Internet = 5	HTTP 422; rechazo de esquema
A07	Label = Ataque-Desconocido	HTTP 422; rechazo de categoría

Registrar para cada prueba: fecha, versión/digest de imagen, URL sin secretos, entrada, código HTTP, respuesta, resultado esperado, responsable y captura. Estas pruebas en Azure son pendientes; los resultados HTTP citados corresponden al entorno Docker local.

```
curl -X POST "https://<FQDN>/predict" -H "Content-Type: application/json" -d
"{\"Label\":\"SSH-
Bruteforce\",\"Criticidad_Activo\":\"Alta\",\"Exposicion_Internet\":1,\"Cuenta_Privilegiada
\":1}"
```

La respuesta esperada contiene prioridad_sugerida = Crítica, identificación del modelo y aviso de supervisión humana. Las diferencias entre CMD, PowerShell y Bash pueden requerir adaptar el escapado de comillas; alternativamente, ejecutar la solicitud desde /docs.

7. Operación, Monitoreo y Revisión

7.1 Observabilidad Mínima

Registrar eventos de inicio, errores de carga del modelo, fallas HTTP y tiempos de respuesta. Revisar el estado de revisiones y logs del contenedor. No registrar en texto plano credenciales ni cargas útiles que contengan datos sensibles. Definir límites de recursos, escalamiento y alertas de costo acordes con el ejercicio.

7.2 Actualización Controlada

Ante una nueva versión, reconstruir y publicar con un tag distinto (por ejemplo, 1.1), actualizar la imagen de Container Apps y repetir los casos A01–A07. Conservar la versión previa para reversión y documentar cambios en Git. Una imagen construida con dependencias fijadas no garantiza ausencia de vulnerabilidades; debe escanearse periódicamente.

7.3 Recuperación y Eliminación de Recursos

```
az containerapp revision list --name ca-faseii-demo --resource-group rg-faseii-demo -o
table
az group delete --name rg-faseii-demo --yes --no-wait
```

La eliminación del grupo de recursos es irreversible para los recursos incluidos. Ejecutarla únicamente tras comprobar que no contiene servicios ajenos a la práctica y conservar las evidencias necesarias. La reversión de una revisión debe efectuarse con los comandos vigentes de Container Apps y permisos autorizados.

8. Herramientas de Documentación y Uso Responsable de la IA

La documentación técnica se estructura con README.md, Dockerfile, requirements.txt, OpenAPI/Swagger en /docs y el presente manual. Git conserva el historial y permite relacionar la documentación con una versión del código. Las capturas de pantalla y las salidas de comandos deben acompañarse de fecha, entorno y resultado observado.

ChatGPT se utilizó como apoyo para organizar pasos, revisar redacción y contrastar riesgos y criterios de prueba. La ejecución de comandos, la comprobación de resultados y la decisión de publicar recursos deben permanecer bajo responsabilidad del usuario. Copilot puede utilizarse opcionalmente para revisar documentación o sugerir comandos, pero en el presente trabajo no fue utilizada esta IA generativa.

Antes de aceptar sugerencias de cualquier herramienta de IA, comprobar su compatibilidad con las versiones instaladas y la documentación oficial. No proporcionar contraseñas, tokens de acceso ni datos operativos reales a herramientas no autorizadas.

9. Resultados, Limitaciones y Conclusiones

La Fase II acreditó localmente la construcción de la imagen Docker, la ejecución del contenedor, el funcionamiento de FastAPI y la respuesta correcta en cuatro escenarios de prioridad y dos validaciones de entradas inválidas. El repositorio GitHub conserva el código, el modelo y el notebook de entrenamiento. La guía propuesta extiende esa arquitectura hacia Azure Container Apps, pero su ejecución y sus resultados remotos quedan pendientes.

El modelo reproduce una política académica de priorización construida con etiquetas de eventos y atributos contextuales sintéticos. Por tanto, una clasificación perfecta en el conjunto académico no prueba efectividad operativa frente a amenazas reales. Para uso institucional serían necesarios controles de autenticación, pruebas de seguridad, datos representativos, supervisión humana efectiva, monitoreo y validación independiente.

Se concluye que la contenerización local aporta una base transferible a un entorno administrado. El valor del manual radica en establecer requisitos, secuencia de despliegue, criterios de aceptación, medidas de seguridad y un procedimiento de limpieza, sin confundir una propuesta técnica con un despliegue efectivamente ejecutado.

Referencias Bibliográficas

Microsoft. (s. f.). Azure Container Apps documentation. https://learn.microsoft.com/azure/container-apps/

Microsoft. (s. f.). Azure Container Registry documentation. https://learn.microsoft.com/azure/container-registry/

Docker Inc. (s. f.). Docker documentation. https://docs.docker.com/

FastAPI. (s. f.). FastAPI documentation. https://fastapi.tiangolo.com/

GitHub. (s. f.). Git documentation. https://docs.github.com/en/get-started/using-git

scikit-learn developers. (s. f.). Model evaluation. https://scikit-learn.org/stable/modules/model_evaluation.html